

```
dotnet add package Azure.Security.KeyVault.Secrets --version 4.8.0
```

```
dotnet add package Azure.Storage.Blobs --version 12.25.0
```

```
using Azure.Identity;
using Azure.Security.KeyVault.Secrets;
using Azure.Storage;
using Azure.Storage.Blobs;
using Azure.Storage.Sas;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.Extensions.Logging;
using System.Net;

namespace Company.FunctionApp2;

public class Function
{
    private readonly ILogger<Function> _logger;

    public Function(ILogger<Function> logger)
    {
        _logger = logger;
    }

    [Function("Function")]
    public async Task<HttpResponseBody> Run(
        [HttpTrigger(AuthorizationLevel.Function, "get", Route = "generate-sas/{blobName}")]
        HttpRequestData req,
        string blobName)
    {
        _logger.LogInformation($"Generating SAS for blob: {blobName}");

        string connectionString =
            Environment.GetEnvironmentVariable("AzureStorageConnectionString");
        string containerName = Environment.GetEnvironmentVariable("ContainerName");
        string keyVaultUri = Environment.GetEnvironmentVariable("KeyVaultUri");

        if (string.IsNullOrEmpty(connectionString) || string.IsNullOrEmpty(containerName) ||
            string.IsNullOrEmpty(keyVaultUri))
        {
            var errorResponse = req.CreateResponse(HttpStatusCode.InternalServerError);
            await errorResponse.WriteStringAsync("Missing required configuration settings.");
            return errorResponse;
        }
    }
}
```

```

// Create BlobServiceClient from connection string
var blobServiceClient = new BlobServiceClient(connectionString);

// Get account name from BlobServiceClient
var accountName = blobServiceClient.AccountName;

// Parse AccountKey safely by splitting (safe if you validate existence)
string accountKey = null;
foreach (var part in connectionString.Split(';'))
{
    if (part.StartsWith("AccountKey=", StringComparison.OrdinalIgnoreCase))
    {
        accountKey = part.Substring("AccountKey=".Length);
        break;
    }
}

if (string.IsNullOrEmpty(accountKey))
{
    var errorResponse = req.CreateResponse(HttpStatusCode.InternalServerError);
    await errorResponse.WriteStringAsync("AccountKey not found in connection
string.");
    return errorResponse;
}

// Create credential
var credential = new StorageSharedKeyCredential(accountName, accountKey);

// Create BlobClient
var blobClient =
blobServiceClient.GetBlobContainerClient(containerName).GetBlobClient(blobName);

// Build SAS
DateTimeOffset expiresOn = DateTimeOffset.UtcNow.AddHours(1);
var sasBuilder = new BlobSasBuilder
{
    BlobContainerName = containerName,
    BlobName = blobName,
    Resource = "b",
    ExpiresOn = expiresOn
};
sasBuilder.SetPermissions(BlobSasPermissions.Read | BlobSasPermissions.Write);

var sasUri = blobClient.GenerateSasUri(sasBuilder);

// Store SAS URL in Key Vault

```

```

        var secretClient = new SecretClient(new Uri(keyVaultUri), new
DefaultAzureCredential());
        string secretName = "";

        var secretToStore = new KeyVaultSecret(secretName, sasUri.ToString())
        {
            Properties =
            {
                Tags = { { "ExpiresOn", expiresOn.UtcDateTime.ToString("o") } }
            }
        };

        await secretClient.SetSecretAsync(secretToStore);

        // Return SAS URL
        var response = req.CreateResponse(HttpStatusCode.OK);
        await response.WriteAsJsonAsync(new
        {
            sasUrl = sasUri.ToString(),
            expiresOn
        });

        return response;
    }
}
-----
{
    "IsEncrypted": false,
    "Values": {
        "AzureWebJobsStorage": "UseDevelopmentStorage=true",
        "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated",
        "AzureWebJobsSecretStorageType": "files",
        "AzureStorageConnectionString": "",
        "KeyVaultUri": "",
        "ContainerName": "images"
    }
}

```

az login

az version

az group create --name reJul25 --location eastus2

```
az storage account create --name mystorageacct20250715 --location eastus2
--resource-group reJul25 --sku Standard_LRS
```

```
az functionapp create --resource-group reJul25 --consumption-plan-location eastus2 --name
mydotnetfunc20250715 --storage-account mystorageacct20250715 --runtime dotnet-isolated
--functions-version 4
```

```
az functionapp config appsettings set --name mydotnetfunc20250715 --resource-group
reJul25 --settings AzureStorageConnectionString="" ContainerName="images"
KeyVaultUri="/"
```

```
az functionapp function keys list --resource-group reJul25 --name mydotnetfunc20250715
--function-name Function
```

```
func azure functionapp logstream mydotnetfunc20250715 --output json
```

```
-- add the function to vault
-- enable identity in function
```

Automatic Regeneration:

If the SAS URL is expired or invalid, the **next time GetBlobClientWithSas is called** (e.g., during upload or download), it will request a new SAS URL from the external endpoint.

The code requests a fresh SAS URL every time it needs to access a blob, ensuring that it always uses a valid, up-to-date SAS token. If the token is expired or invalid, the regeneration happens automatically via the HTTP request to the function endpoint.

Azure CLI:

- az login
- az group create --name reJul25 --location eastus2
- az storage account create --name mystorageacct20250715 --location eastus2 --resource-group reJul25 --sku Standard_LRS
- az functionapp create --resource-group reJul25 --consumption-plan-location eastus2 --name mydotnetfunc20250715 --storage-account mystorageacct20250715 --runtime dotnet-isolated --functions-version 4
- az functionapp config appsettings set --name mydotnetfunc20250715 --resource-group reJul25 --settings AzureStorageConnectionString="" ContainerName="images" KeyVaultUri="/"
-

Steps to implement that functions

1. Install the azure resources, azure functions —> under extensions
2. So it will share the icon of the azure, click that -> if incase no login, make sure to login, otherwise under workspace you will see a **Create Azure Functions** so click that and create a new one with a trigger (HTTP) and name
3. After creation create the [Function.cs](#) in the above and put make sure to replace the correct secret variable that is created under keyvault
4. In local.setting.json put, AzureWebJobsSecretStorageType, AzureStorageConnectionString, KeyVaultUri, ContainerName
5. In side the folder check the dotnet restore, func start and check the api given in the terminal and test
6. Right click the folder it shows **Deploy Function App** , after it shows the function group name and data and finally click **Deploy in the prompt**

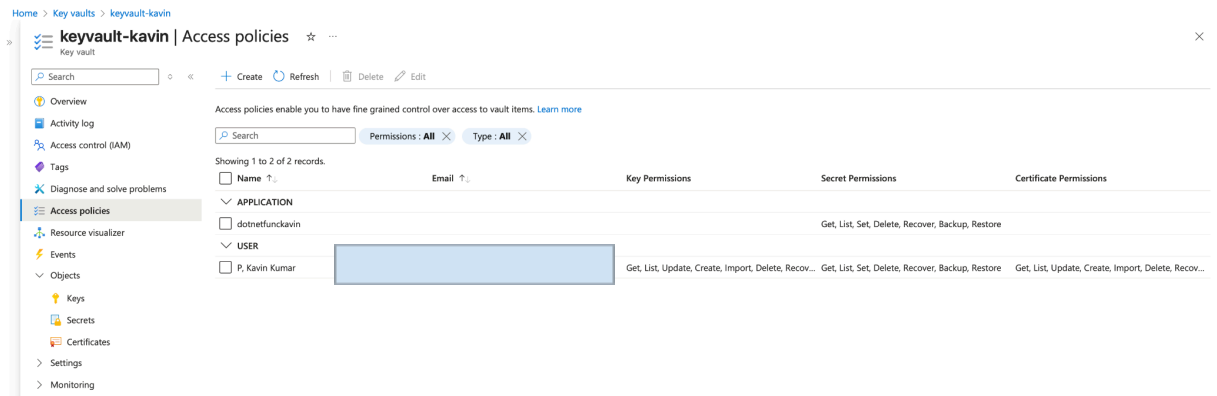
In the azure dashboard you will get the link

```
https://<>.net/api/generate-sas/{fileName}?code==
```

And the code is get by using the **command in CLI**

```
az functionapp function keys list --resource-group rg-name --name  
dotnetfunc20250715 --function-name Function
```

- After that go to the Keyvault



- Under access policy create a new one and in the
- In the configure template drop down select **Secret Management** and click next
- in the next page **select or search the function app names**
- Create create
- After **enable** the Identity option -> **on** option in the **function APP**
 - Present inside the created one
 - **Make the status as ON and save**

Check the Url in the postman for receiving files or not

- **If yes**, then implement that in the blobstorage backend api, so every time when we call any of the api, it will generate a new sas and store in vault and get the result